

Ticket: Build a Payment Processing System

Difficulty:  Intermediate / Advanced

Type: Backend / OOP

Goal: Implement the missing classes and methods in an existing Python codebase.

Scenario

You're working on an e-commerce application.

The checkout system needs to support multiple payment methods:

- Credit card
- PayPal
- Bank transfer

The application should be designed so that adding a new payment method later doesn't require rewriting the checkout system.

Your job is to complete the unfinished code.

Project structure

```
payment_system/  
|  
├── models.py  
├── interfaces.py  
├── payments.py  
├── checkout.py  
├── exceptions.py  
└── main.py
```

models.py

```
from dataclasses import dataclass
from decimal import Decimal
from enum import Enum
```

```
class PaymentStatus(Enum):
    PENDING = "pending"
    SUCCESS = "success"
    FAILED = "failed"
```

```
@dataclass
class PaymentRequest:
    order_id: str
    amount: Decimal
    currency: str
    customer_email: str
```

```
@dataclass
class PaymentResult:
    status: PaymentStatus
    transaction_id: str | None = None
    message: str | None = None
```

Don't modify the models unless you discover a genuine design problem.

exceptions.py

```
class PaymentError(Exception):
    pass
```

```
class InvalidPaymentRequest(PaymentError):
    pass
```

```
class PaymentProcessingError(PaymentError):  
    pass
```

```
class UnsupportedCurrency(PaymentError):  
    pass
```

interfaces.py

Your first task is to define the payment processor interface.

```
from abc import ABC, abstractmethod
```

```
from models import PaymentRequest, PaymentResult
```

```
class PaymentProcessor(ABC):
```

```
    @abstractmethod
```

```
    def process(self, request: PaymentRequest) -> PaymentResult:  
        pass
```

```
    @abstractmethod
```

```
    def refund(self, transaction_id: str) -> bool:  
        pass
```

```
    @property
```

```
    @abstractmethod
```

```
    def supported_currencies(self) -> set[str]:  
        pass
```

The interface is already mostly defined.

Do not implement payment logic here.

payments.py

You need to implement these classes.

```
from models import PaymentRequest, PaymentResult, PaymentStatus
from interfaces import PaymentProcessor
from exceptions import (
    InvalidPaymentRequest,
    PaymentProcessingError,
    UnsupportedCurrency,
)
```

```
class CreditCardProcessor(PaymentProcessor):

    def __init__(self):
        # TODO
        pass

    @property
    def supported_currencies(self) -> set[str]:
        # TODO
        pass

    def process(self, request: PaymentRequest) -> PaymentResult:
        # TODO
        pass

    def refund(self, transaction_id: str) -> bool:
        # TODO
        pass
```

```
class PayPalProcessor(PaymentProcessor):

    def __init__(self):
        # TODO
        pass

    @property
    def supported_currencies(self) -> set[str]:
```

```

        # TODO
        pass

    def process(self, request: PaymentRequest) -> PaymentResult:
        # TODO
        pass

    def refund(self, transaction_id: str) -> bool:
        # TODO
        pass

class BankTransferProcessor(PaymentProcessor):

    def __init__(self):
        # TODO
        pass

    @property
    def supported_currencies(self) -> set[str]:
        # TODO
        pass

    def process(self, request: PaymentRequest) -> PaymentResult:
        # TODO
        pass

    def refund(self, transaction_id: str) -> bool:
        # TODO
        pass

```

Business rules

This is where the actual challenge begins.

General validation

Every payment processor must reject:

- Empty `order_id`

- Empty `customer_email`
- $\text{Amount} \leq 0$
- Empty currency

For example:

`amount = -10`

should not be processed.

Currency support

Each payment processor supports different currencies.

You decide the exact currencies, but they must differ between processors.

If a request uses an unsupported currency, raise:

`UnsupportedCurrency`

Credit card

Credit-card payments should:

- Validate the payment request.
- Generate a transaction ID.
- Store successful transactions internally.
- Return `PaymentStatus.SUCCESS`.
- Allow successful transactions to be refunded.
- Reject refunds for unknown transaction IDs.

PayPal

PayPal should behave similarly, but use its own transaction storage.

It should **not share transaction state with the credit-card processor**.

Bank transfer

Bank transfers behave slightly differently.

A bank transfer should initially return:

`PaymentStatus.PENDING`

instead of `SUCCESS`.

You should decide how the transaction should be represented internally.

checkout.py

Now implement the service that uses the interface.

```
from models import PaymentRequest, PaymentResult
from interfaces import PaymentProcessor
from exceptions import PaymentError
```

```
class CheckoutService:
```

```
    def __init__(self, processors: list[PaymentProcessor]):
        # TODO
        pass
```

```
    def pay(
        self,
        processor_name: str,
        request: PaymentRequest,
    ) -> PaymentResult:
        # TODO
        pass
```

```
    def refund(
        self,
        processor_name: str,
        transaction_id: str,
    ) -> bool:
```

```
# TODO
pass
```

Important requirement

CheckoutService **must not contain code like this**:

```
if processor_name == "credit_card":
    ...
elif processor_name == "paypal":
    ...
elif processor_name == "bank_transfer":
    ...
```

The whole point is that CheckoutService should work with the PaymentProcessor abstraction.

If you later create:

```
class ApplePayProcessor(PaymentProcessor):
    ...
```

you should be able to add it without rewriting the checkout logic.

main.py

You should be able to use the finished system roughly like this:

```
from decimal import Decimal

from models import PaymentRequest
from payments import (
    CreditCardProcessor,
    PayPalProcessor,
    BankTransferProcessor,
)
from checkout import CheckoutService
```

```
processors = [  
    CreditCardProcessor(),  
    PayPalProcessor(),  
    BankTransferProcessor(),  
]  
  
checkout = CheckoutService(processors)  
  
request = PaymentRequest(  
    order_id="ORDER-123",  
    amount=Decimal("49.99"),  
    currency="EUR",  
    customer\_email="customer@example.com",  
)  
  
result = checkout.pay(  
    "credit_card",  
    request,  
)  
  
print(result)
```

You need to make this work.



Your testing task

Create tests for at least these situations:

Valid payments

Credit card → successful payment

PayPal → successful payment

Bank transfer → pending payment

Invalid payments

Negative amount
Zero amount
Empty order ID
Empty email
Unsupported currency
Unknown payment processor

Refunds

Refund existing credit-card transaction
Refund existing PayPal transaction
Refund unknown transaction
Refund same transaction twice

Architecture

Test that:

```
CheckoutService
    ↓
PaymentProcessor
    ↓
CreditCardProcessor
PayPalProcessor
BankTransferProcessor
```

The checkout service should depend on the **abstraction**, not the concrete implementations.

Bonus requirements

Once the basic version works, add these without changing the existing public API.

Bonus 1 — Transaction repository

Create:

```
class TransactionRepository:  
    ...
```

Move transaction storage out of the payment processors.

Bonus 2 — Logging

Add logging for:

```
Payment started  
Payment succeeded  
Payment failed  
Refund started  
Refund succeeded  
Refund failed
```

Don't log sensitive customer/payment information.

Bonus 3 — New payment method

Without modifying CheckoutService, implement:

```
class ApplePayProcessor(PaymentProcessor):  
    ...
```

It should support its own currencies and transaction handling.

Bonus 4 — Dependency injection

Instead of creating repositories/loggers inside your processors, inject them:

```
CreditCardProcessor(repository, logger)
```

Now your classes should be much easier to unit test.

Your actual assignment

Don't implement everything at once.

Work in this order:

- ☐ Implement `PaymentProcessor` interface
- ☐ Implement `CreditCardProcessor`
- ☐ Implement `PayPalProcessor`
- ☐ Implement `BankTransferProcessor`
- ☐ Implement `CheckoutService`
- ☐ Handle validation
- ☐ Handle unsupported currencies
- ☐ Handle refunds
- ☐ Write unit tests
- ☐ Add repository
- ☐ Add logging
- ☐ Add `ApplePayProcessor`
- ☐ Refactor using dependency injection